AWS LIFT AND SWIFT PROJECT

Project Overview – Lift and Shift Application Workload on AWS

This project focuses on migrating the **VProfile multi-tier web application** from a traditional data center setup to the **AWS Cloud** using a **lift-and-shift strategy**.

In our earlier setup, the application was hosted on local virtual machines using Vagrant. While functional, it required significant manual effort, multiple teams, and ongoing maintenance costs to manage services like Tomcat, MySQL, RabbitMQ, Memcache, and DNS. Scaling up or down was complex and time-consuming.

By moving this workload to AWS, we gain:

- **Scalability** – automatic scale-in and scale-out using Auto Scaling.
- **Cost efficiency** – pay-as-you-go pricing model, no upfront hardware costs.
- **Reliability & Security** – services like Elastic Load Balancer, IAM, ACM for certificates, and Route 53 DNS.
- **Automation** – Infrastructure as Code and streamlined deployment processes.

The new AWS architecture includes:

- **EC2 instances** for application and backend services.
- **Elastic Load Balancer (ALB)** for secure traffic distribution.
- **Auto Scaling Groups** to adjust capacity based on demand.
- **Amazon S3 / EFS** for artifact and data storage.
- **Route 53 Private DNS** for service discovery.
- **ACM** for SSL/TLS certificates.

Objective:
To modernize the VProfile application by running it on AWS Cloud with flexible, automated, and cost-efficient infrastructure, ensuring smoother operations and easier scaling compared to a traditional data center setup.

Flow of Execution – Lift and Shift on AWS

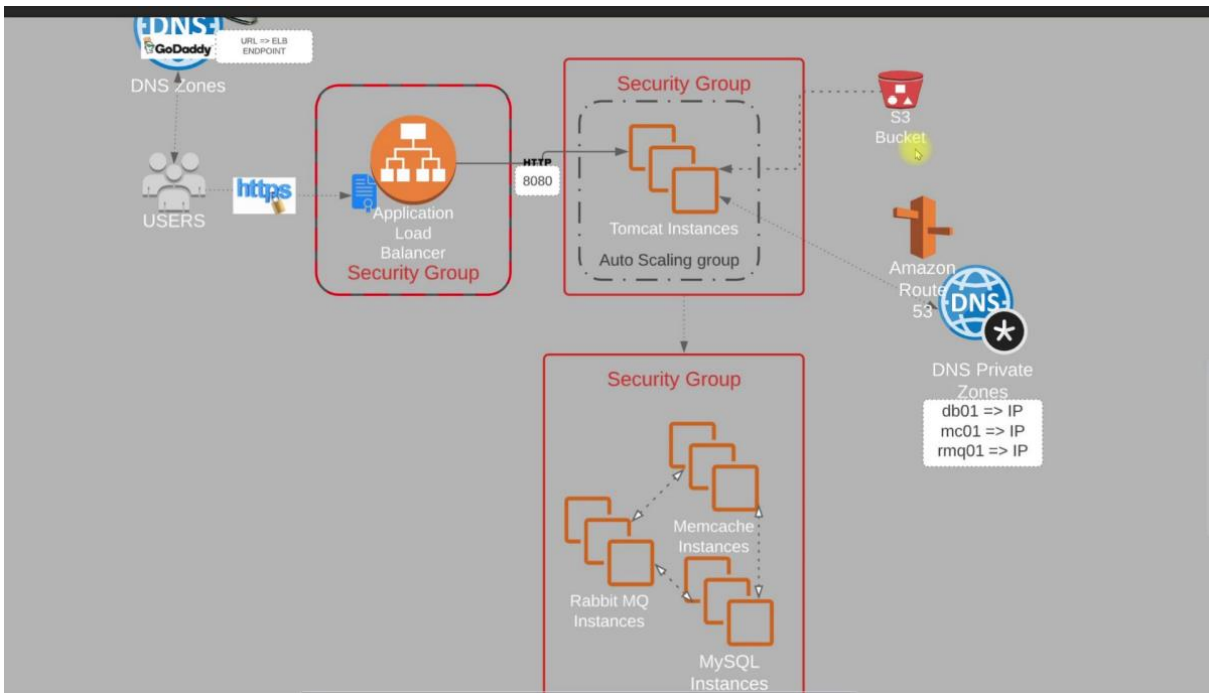
1. **Login & Setup**
    - Log in to AWS account.
    - Create **key pairs** for secure EC2 access.
    - Define **security groups** for Load Balancer, Tomcat, and backend servers.
2. **Launch Application Servers**
    - Provision EC2 instances with **user data scripts** for setup.
    - Configure backend services (MySQL, RabbitMQ, Memcache).
    - Register backend server names and private IPs in **Route 53 Private DNS**.
3. **Build & Deploy Application**
    - Build the **VProfile application artifact** locally.
    - Upload the artifact to an **S3 bucket**.
    - Deploy the artifact from S3 to Tomcat EC2 instances.
4. **Set Up Load Balancer**

- Create an **Application Load Balancer (ALB)**.

5. **Enable Auto Scaling**

   - Configure **Auto Scaling Groups** for Tomcat servers.
   - Automatically adjust instance count based on traffic load.

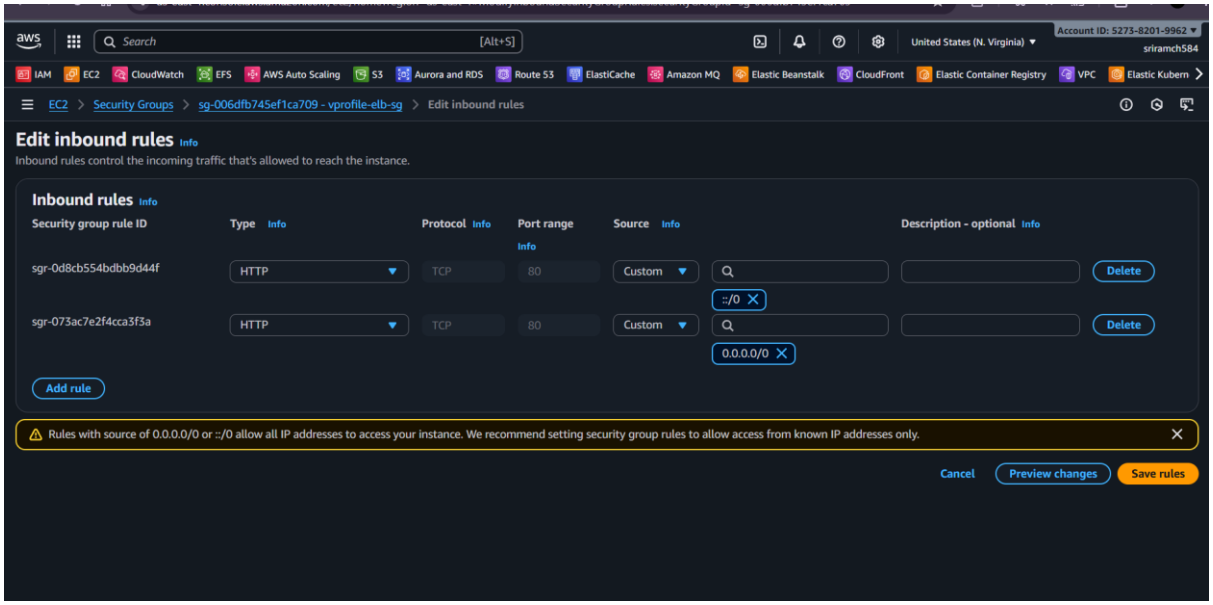
Architecture:



🔒 Security Groups Setup – Summary:

. Load Balancer Security Group (vprofile-elb-sg)

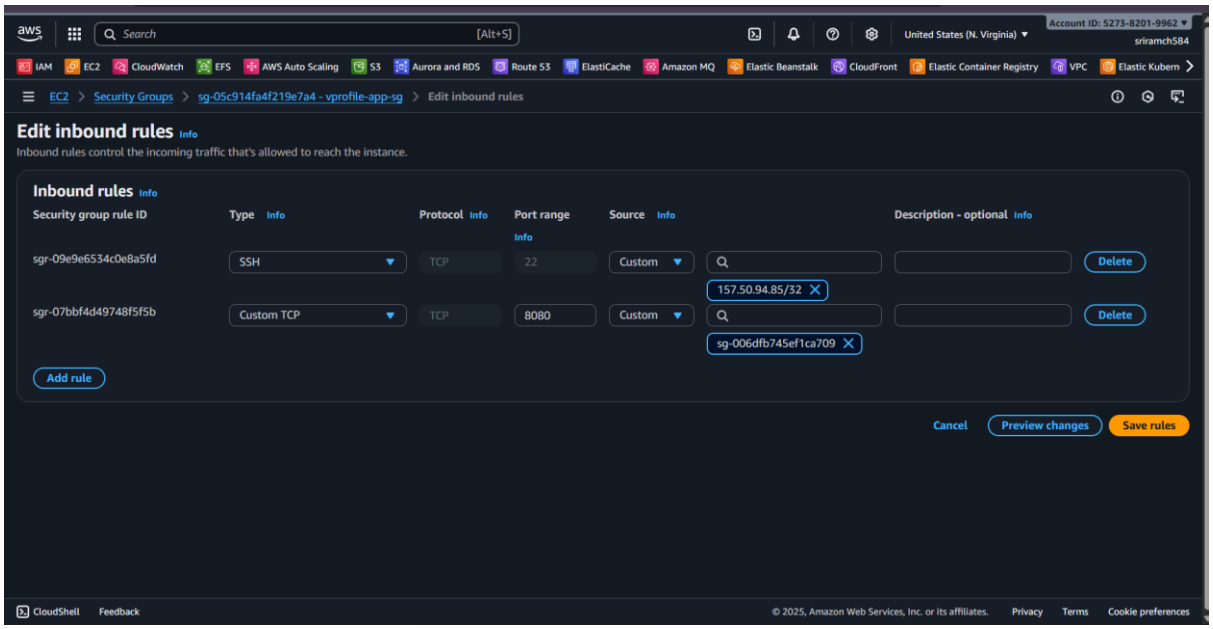
- Handles **internet traffic** (public entry point).
- **Inbound Rules:**
  - **HTTP (80)** – allowed from anywhere (public access).
- **Outbound Rules:** Default (allow all).



---

2. Application / Tomcat Security Group (vprofile-app-sg)

- Protects the **Tomcat servers** running the VProfile application.
- **Inbound Rules:**
  - **8080** – only from the Load Balancer SG.
  - **22 (SSH)** – only from My IP (for administration).
- **Outbound Rules:** Default (allow all).

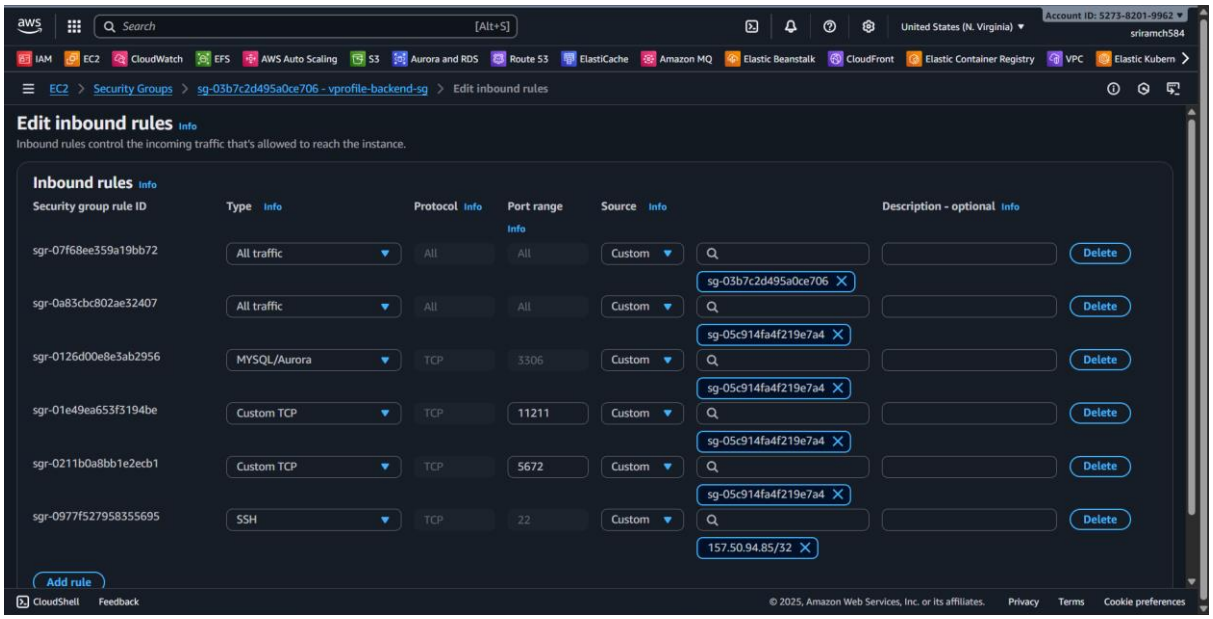


---

3. Backend Services Security Group (vprofile-backend-sg)

- Protects backend servers (**MySQL, Memcache, RabbitMQ**).

- **Inbound Rules:**
  - **3306 (MySQL)** – only from Tomcat SG.
  - **11211 (Memcache)** – only from Tomcat SG.
  - **5672 (RabbitMQ)** – only from Tomcat SG.
  - **22 (SSH)** – only from My IP.
  - **Self-reference rule** – allow all traffic within backend SG (so backend instances can communicate with each other).
- **Outbound Rules:** Default (allow all).



🚀 Summary – Setting Up EC2 Instances with User Data

🎯 Goal

Provision **4 EC2 instances** (MySQL, Memcache, RabbitMQ, Tomcat) using **user data scripts** to automatically install and configure services.

📄 Instances & Security Groups

- **MySQL, Memcache, RabbitMQ → Backend SG**
- **Tomcat → App SG**
- (Later) **Load Balancer → ELB SG**

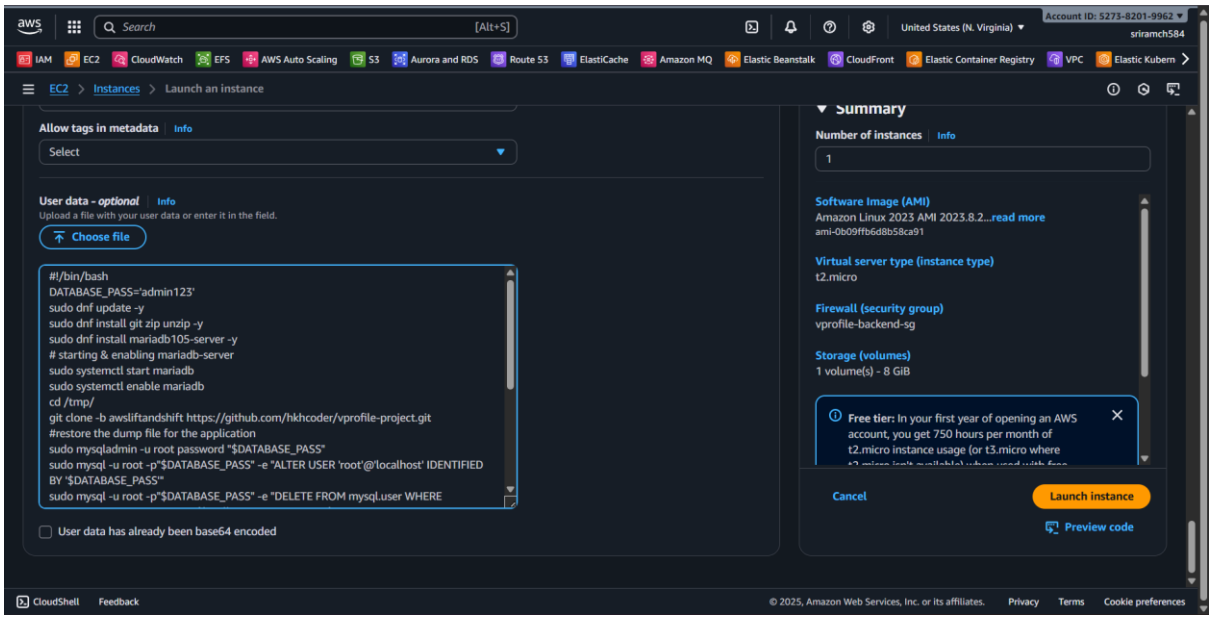
⚙ Steps

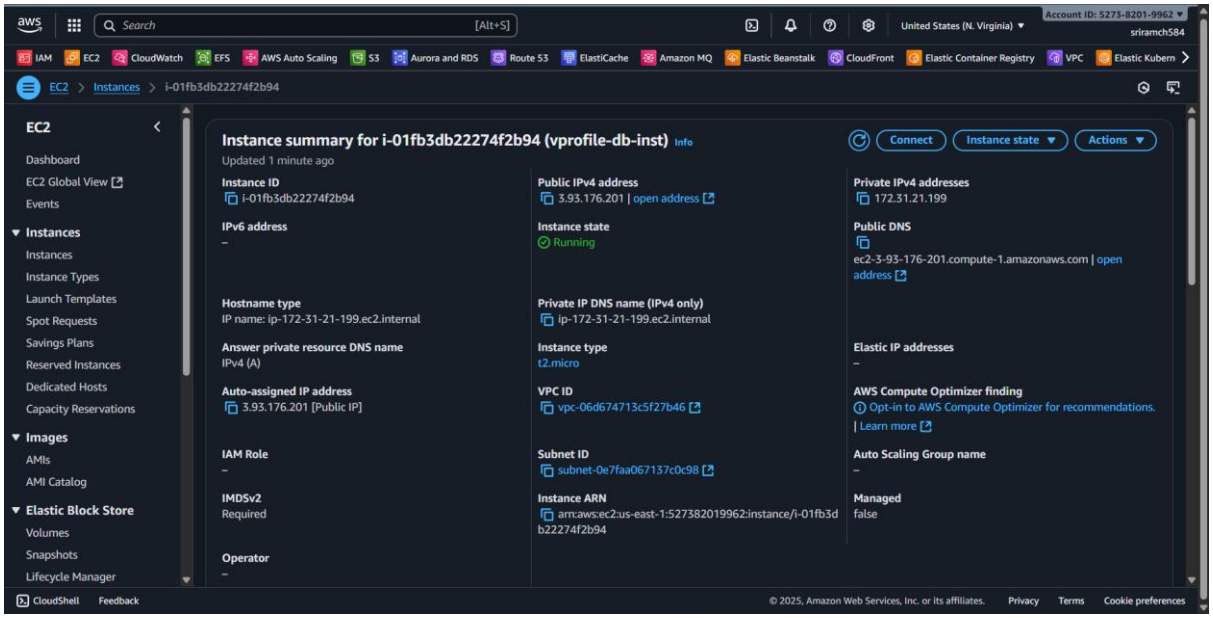
1. Clone Source Code

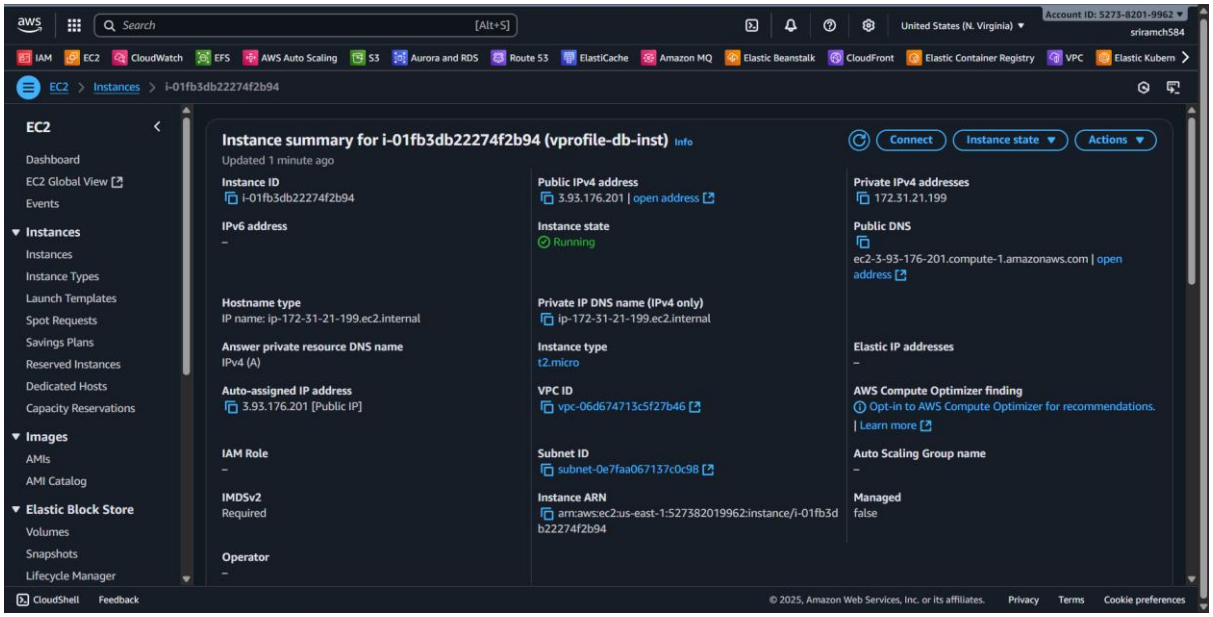
- Clone repo from GitHub → `aws-lift-and-shift` branch.
- Inside repo → `UserData/` folder contains scripts.

2. MySQL Instance (DB01)

- AMI: **Amazon Linux 2023**
- Security Group: **Backend SG**
- User data script:
  - Install **MariaDB (mariadb105-server)**
  - Start & enable service
  - Run SQL commands (secure install, remove test users, create DB & user, grant privileges)
  - Deploy schema (`accounts` DB)
- Verify:
  - `systemctl status mariadb` → running
  - `mysql -u admin -p` → check DB & tables

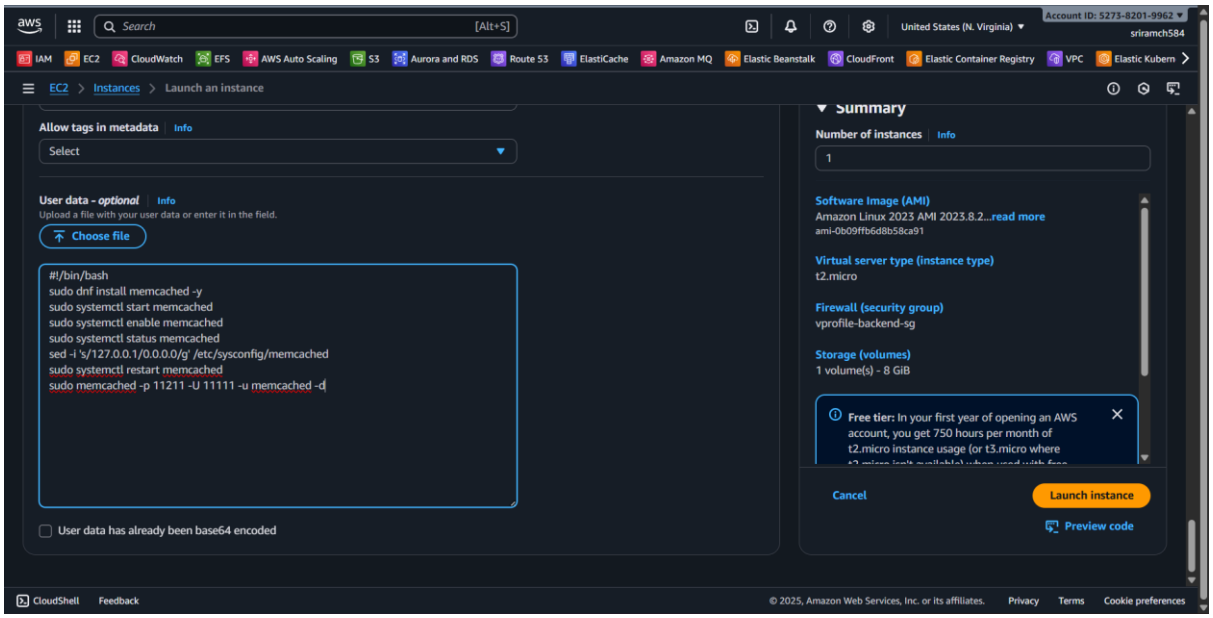


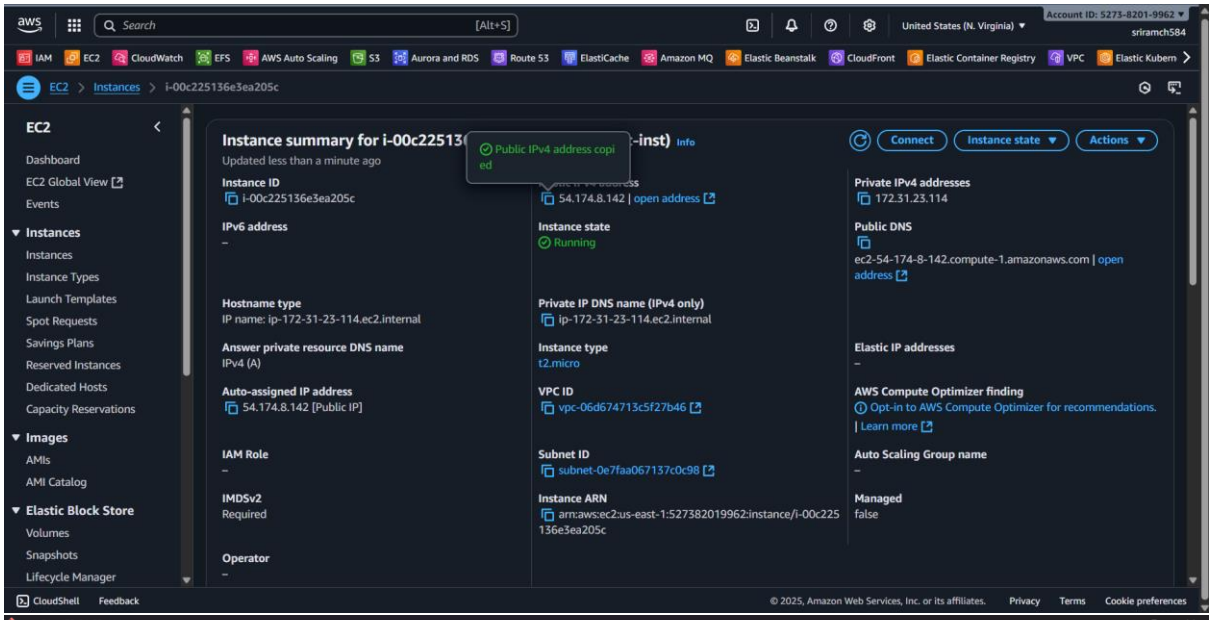


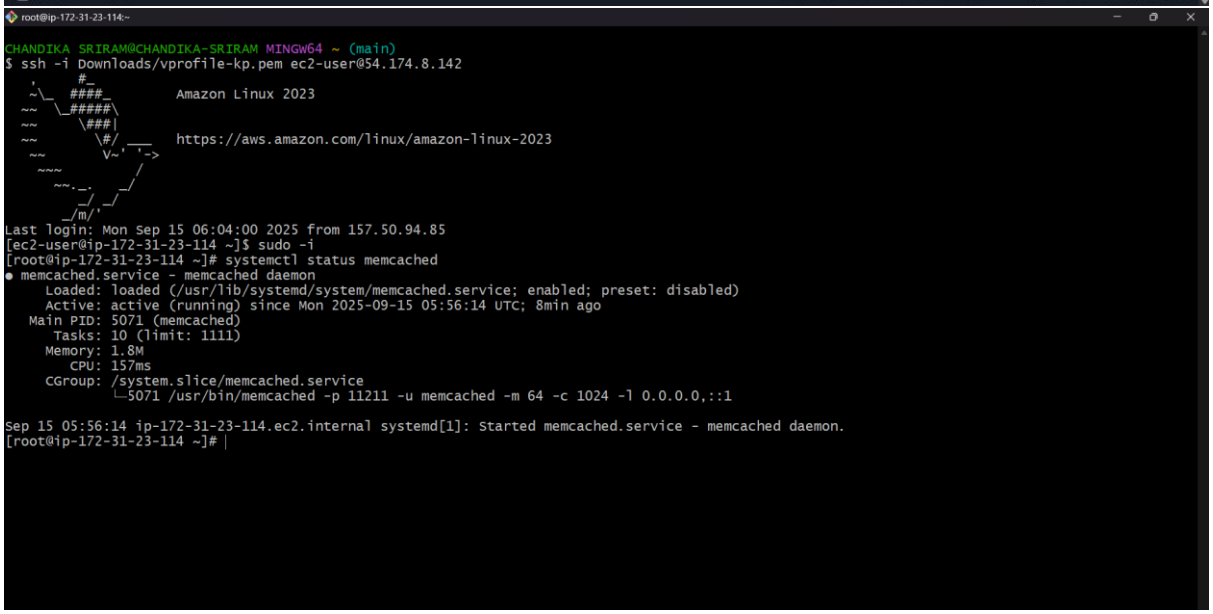


3. Memcache Instance (MC01)

- AMI: **Amazon Linux 2023**
- Security Group: **Backend SG**
- User data script:
  - Install & enable Memcache
  - Update config to listen on **0.0.0.0** (remote access)
- Verify:
  - `systemctl status memcached` → running
  - Listening on port **11211**

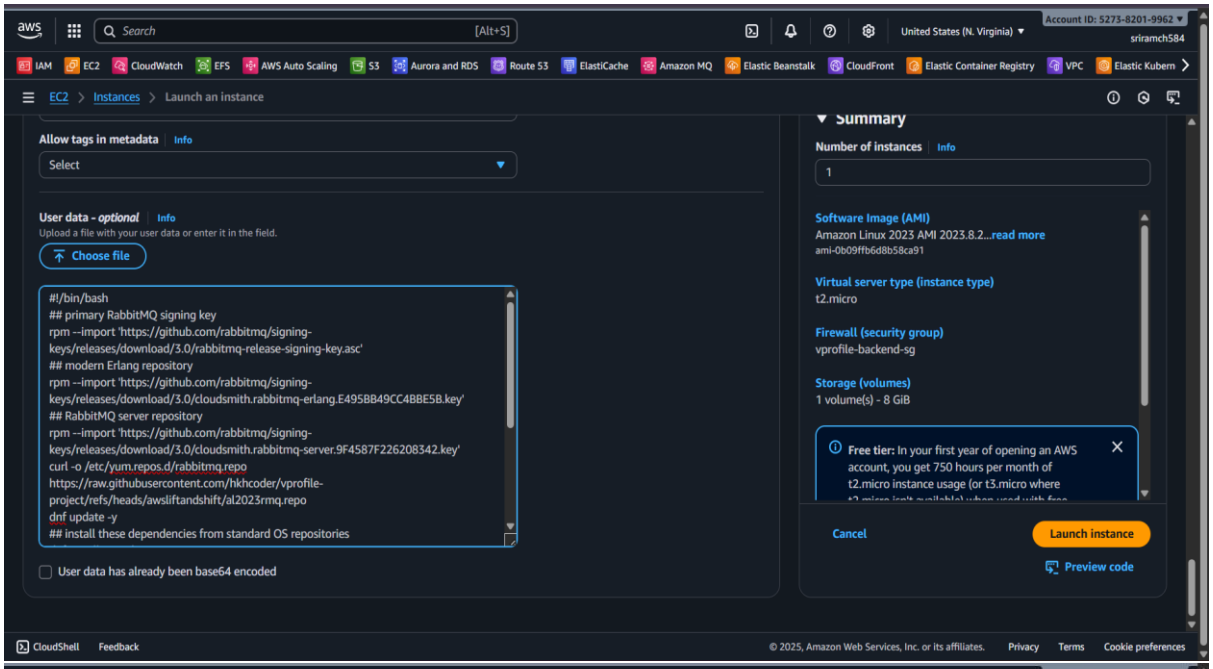


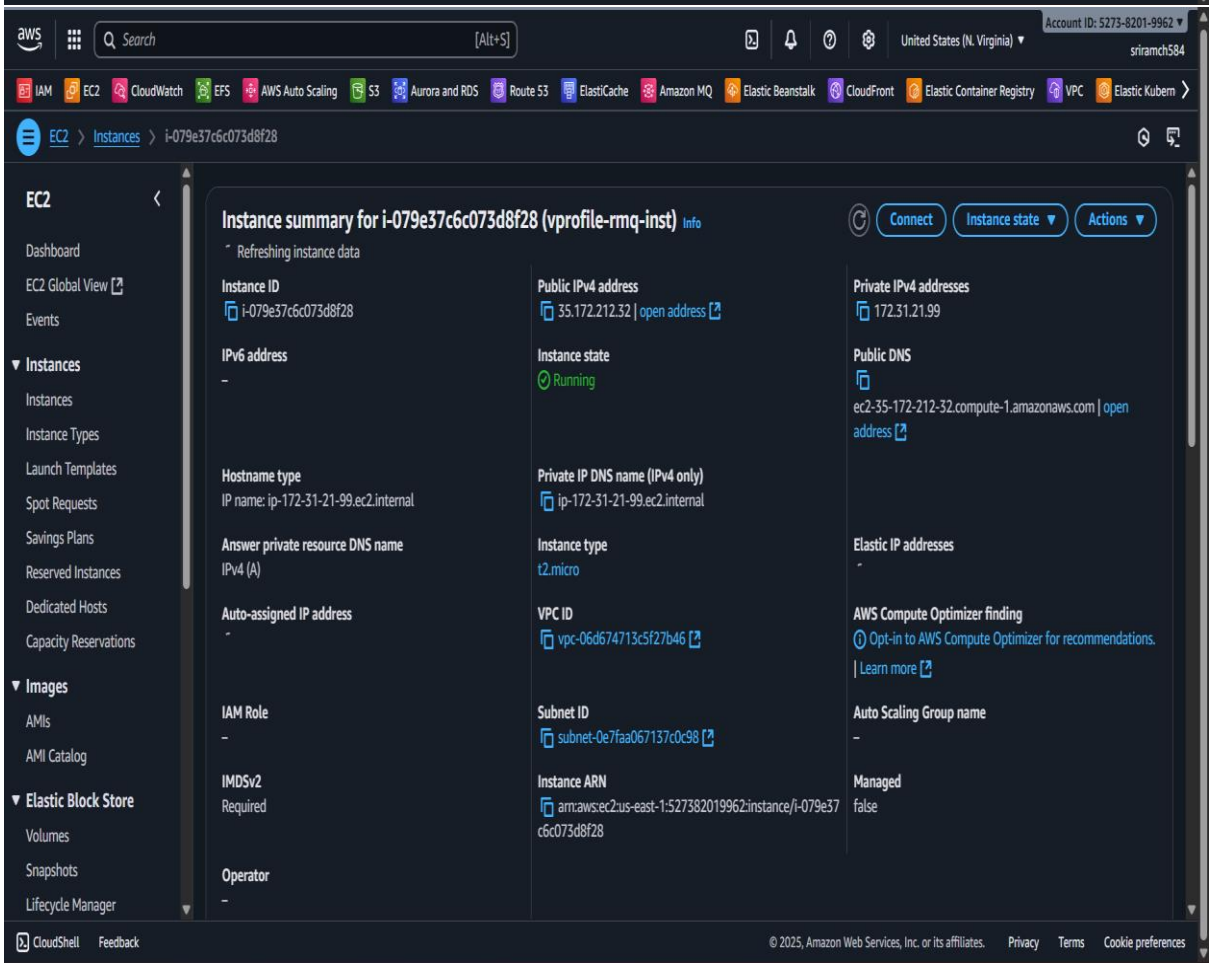


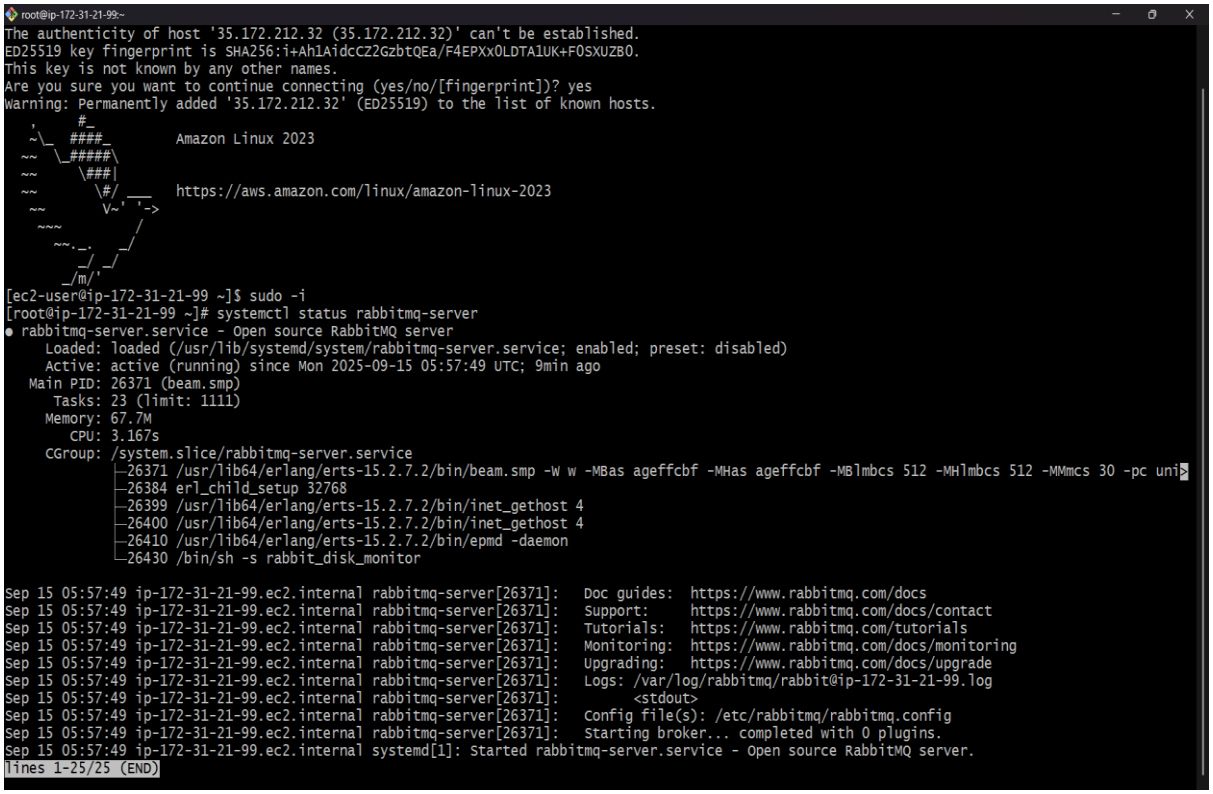


4. RabbitMQ Instance (MQ01)

- AMI: **Amazon Linux 2023**
- Security Group: **Backend SG**
- User data script:
  - Import RabbitMQ & Erlang repository keys
  - Download repo config file from GitHub
  - `dnf install erlang rabbitmq-server`
  - Start & enable RabbitMQ service
  - Add test user, permissions, restart service
- Verify:
  - `systemctl status rabbitmq-server` → running

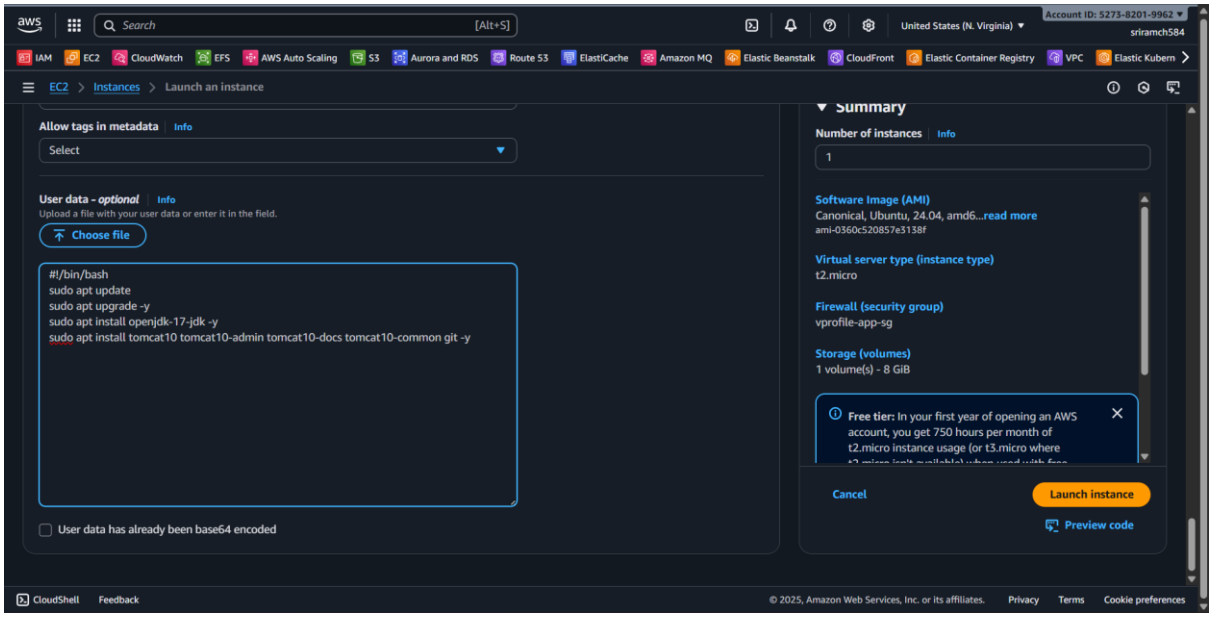

Search [Alt+S]
United States (N. Virginia)
Account ID: 5273-8201-9962
sriramch584
EC2 > Instances > Launch an instance
Allow tags in metadata Info
Select
User data - optional Info
Upload a file with your user data or enter it in the field.
Choose file
#!/bin/bash
primary RabbitMQ signing key
rpm --import 'https://github.com/rabbitmq/signing-keys/releases/download/3.0/rabbitmq-release-signing-key.asc'
modern Erlang repository
rpm --import 'https://github.com/rabbitmq/signing-keys/releases/download/3.0/cloudsmith.rabbitmq-erlang.E495BB49CC4BBE5B.key'
RabbitMQ server repository
rpm --import 'https://github.com/rabbitmq/signing-keys/releases/download/3.0/cloudsmith.rabbitmq-server.9F4587F226208342.key'
curl -o /etc/yum.repos.d/rabbitmq.repo
https://raw.githubusercontent.com/hkhcoder/vprofile-project/refs/heads/awsliftandshift/al2023rmq.repo
dnf update -y
install these dependencies from standard OS repositories
User data has already been base64 encoded
Summary
Number of instances Info
1
Software Image (AMI)
Amazon Linux 2023 AMI 2023.8.2...read more
ami-0b09ffb6d8b58ca91
Virtual server type (instance type)
t2.micro
Firewall (security group)
vprofile-backend-sg
Storage (volumes)
1 volume(s) - 8 GiB
Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where
Cancel
Launch instance
Preview code
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

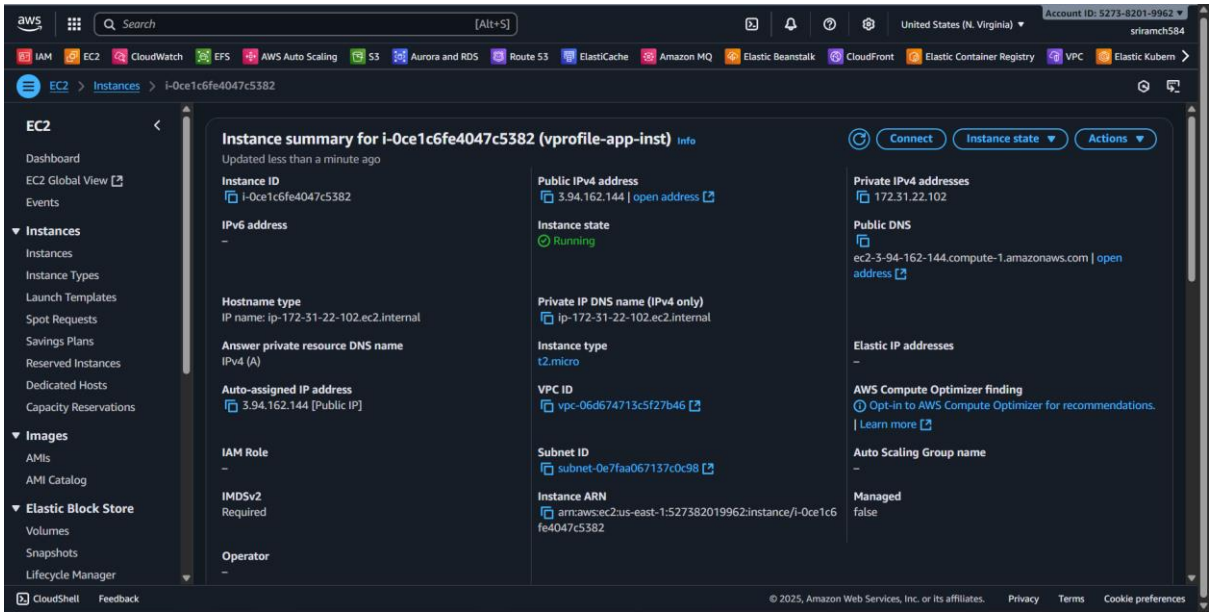

Search [Alt+S]
United States (N. Virginia)
Account ID: 5273-8201-9962
sriramch584
EC2 > Instances > i-079e37c6c073d8f28
EC2
Dashboard
EC2 Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Instance summary for i-079e37c6c073d8f28 (vprofile-rmq-inst) Info
Refreshing instance data
Connect
Instance state
Actions
Instance ID
i-079e37c6c073d8f28
Public IPv4 address
35.172.212.32 | open address
Private IPv4 addresses
172.31.21.99
IPv6 address
-
Instance state
Running
Public DNS
ec2-35-172-212-32.compute-1.amazonaws.com | open address
Hostname type
IP name: ip-172-31-21-99.ec2.internal
Private IP DNS name (IPv4 only)
ip-172-31-21-99.ec2.internal
Answer private resource DNS name
IPv4 (A)
Instance type
t2.micro
Elastic IP addresses
Auto-assigned IP address
VPC ID
vpc-06d674713c5f27b46
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
Learn more
IAM Role
-
Subnet ID
subnet-0e7faa067137c0c98
Auto Scaling Group name
-
IMDSv2
Required
Instance ARN
arn:aws:ec2:us-east-1:527382019962:instance/i-079e37c6c073d8f28
Managed
false
Operator
-
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

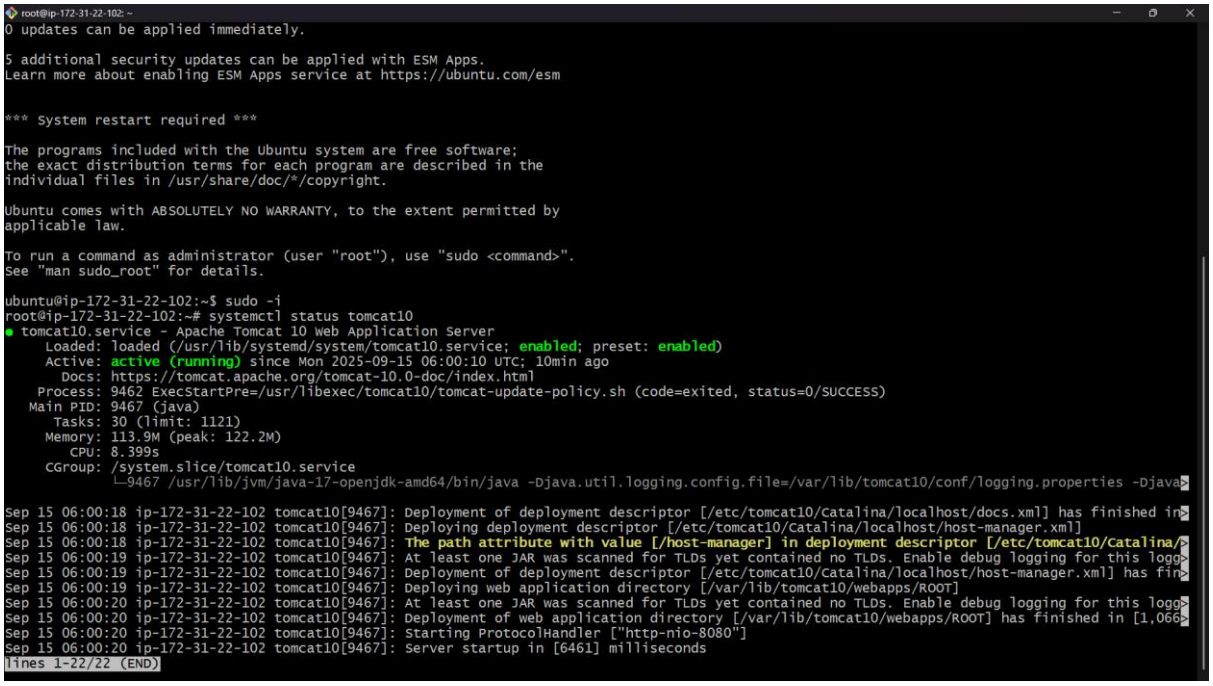


5. Tomcat Instance (APP01)

- AMI: **Ubuntu 24.04 LTS**
- Security Group: **App SG**
- User data script:
  - `apt install tomcat10` (simpler than CentOS setup)
  - Service auto-starts







DNS – Route 53

Problem

- The V-Profile app (**app01**) needs to connect to backend services: **Database (DB01), Memcache (MC01), RabbitMQ (RMQ01)**.
- Config (`application.properties`) uses hostnames like `db01`.
- Hostnames must resolve to **IP addresses** for connectivity.

Issue with using IPs directly:

- Instance recreation changes private IPs.
- Updating config manually is **not maintainable**.
- **Best practice:** use **names**, not IPs, in configuration.

AWS Solution: Route 53

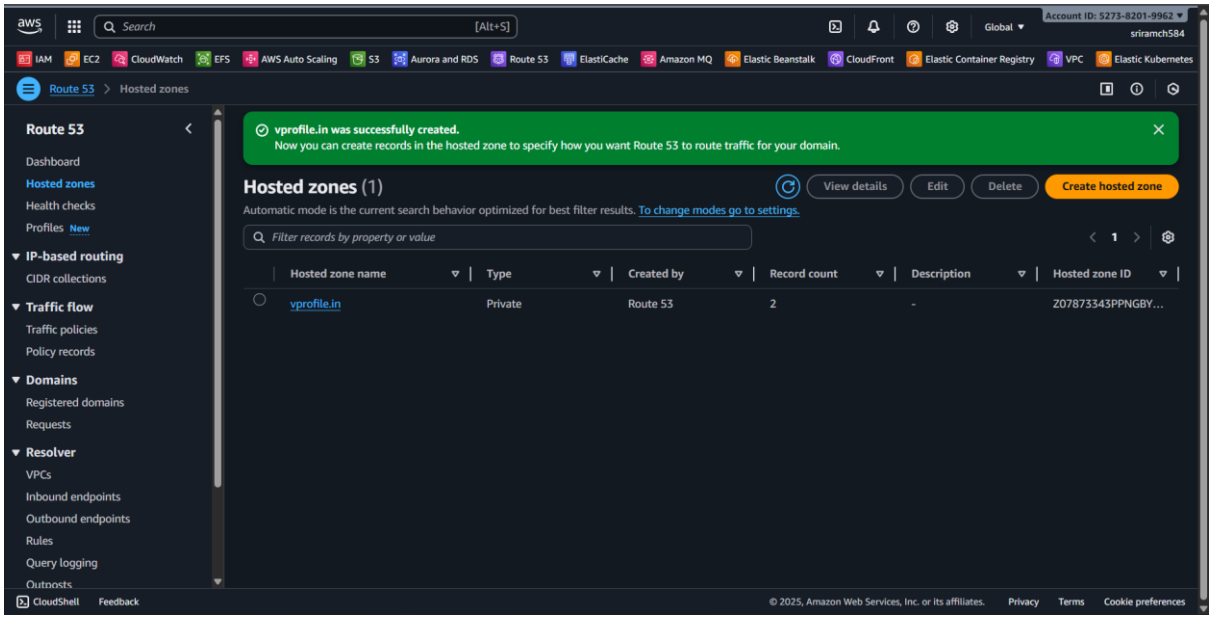
- **Route 53** is AWS DNS.
- Use **Private Hosted Zone** for internal resolution.

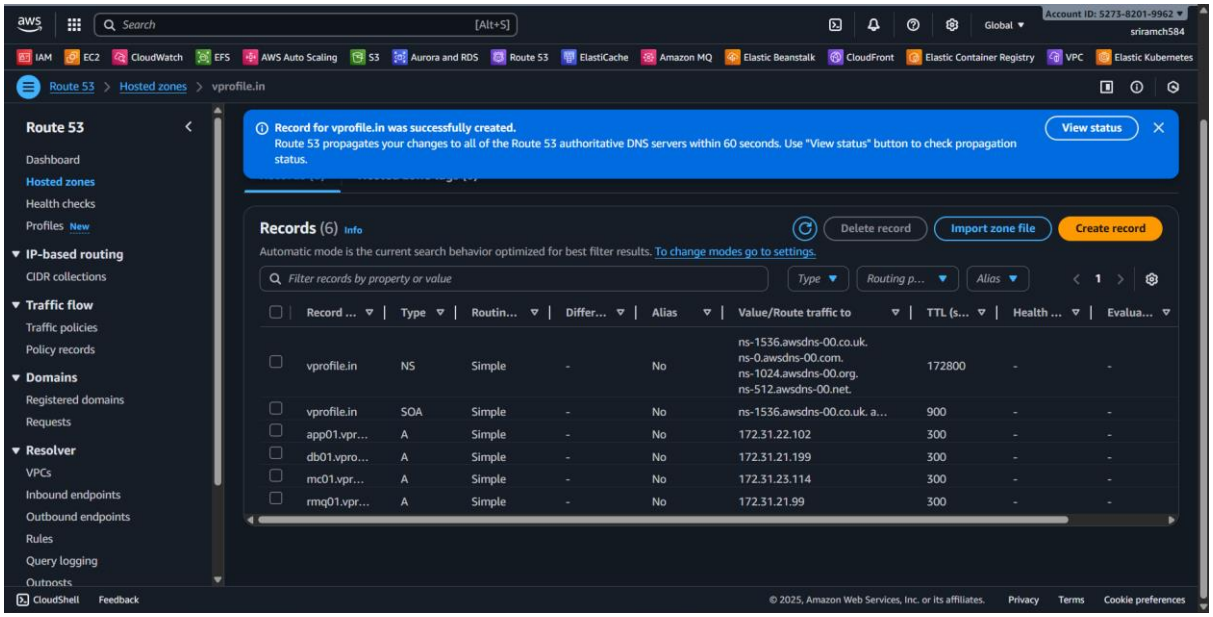
Steps:

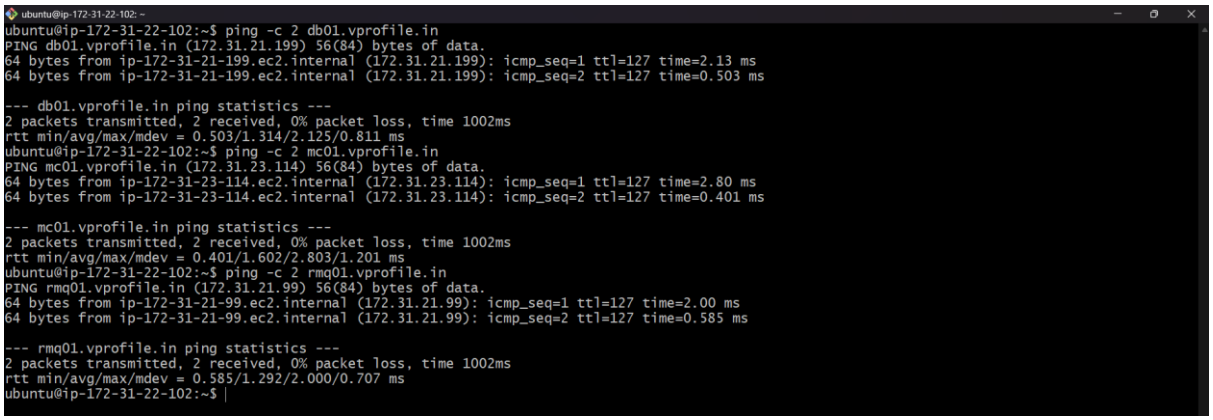
1. **Create Private Hosted Zone**:
   - Domain: `vprofile.in`
   - Associate with **VPC**.
2. **Create A Records**:
   - `db01.vprofile.in` → DB01 private IP
   - `mc01.vprofile.in` → MC01 private IP
   - `rmq01.vprofile.in` → RMQ01 private IP
   - Optional: `app01.vprofile.in` → app01 private IP
3. Use **private IPs**, not public IPs.

Validation

- SSH into **app01** instance.
- Test name resolution:
- `ping -c 4 db01.vprofile.in`
- Ensure resolved IP = actual private IP.
- Repeat for all backend services.







Build and Deploy Artifact

Overview

- Build the **artifact** from source code using **Maven**.
- Push it to **S3 bucket**.
- Fetch it from **app01 Tomcat instance** and deploy it.

Step 1: Prerequisites

On your computer:

- **Java JDK** (version 17) – required by Maven.
- **Maven** (version 3.9.9) – builds the artifact.
- **AWS CLI** – communicate with S3 bucket.

AWS setup needed:

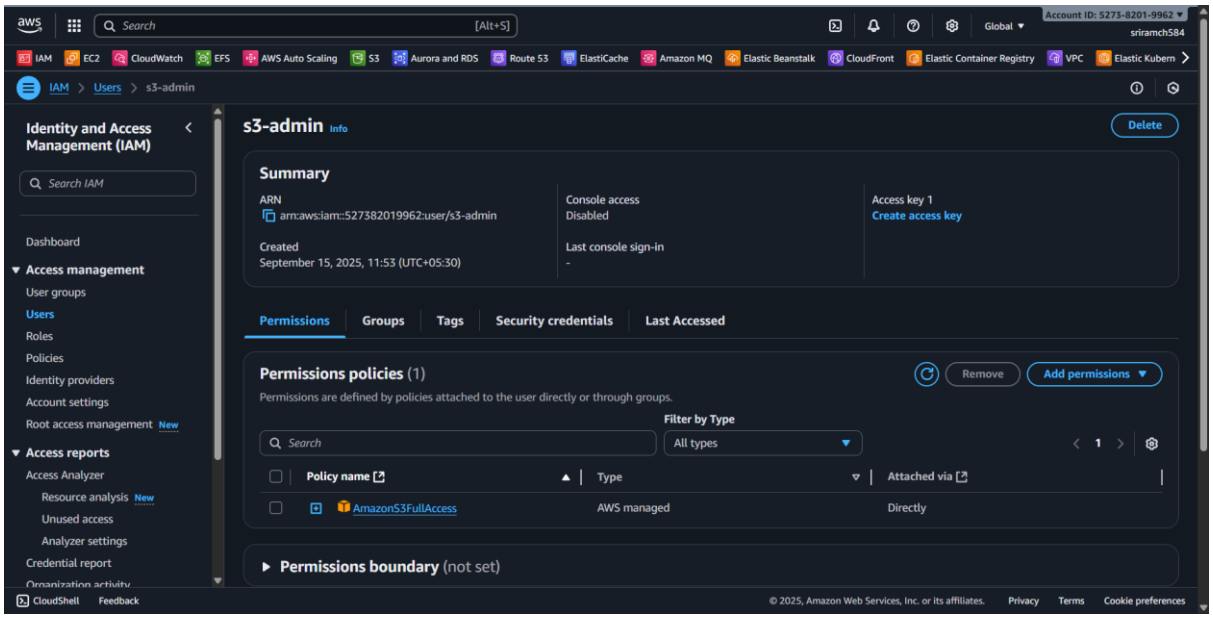
- **S3 bucket** – stores the artifact.
- **IAM user** – provides access keys for authentication from your computer.
- **IAM role** – attached to EC2 instance (app01) to allow S3 access without embedding keys in instance.

Step 2: AWS Configuration

1. **Create S3 bucket**
    - Example: `vprofile-artifact1224`
    - Bucket name must be **unique**.

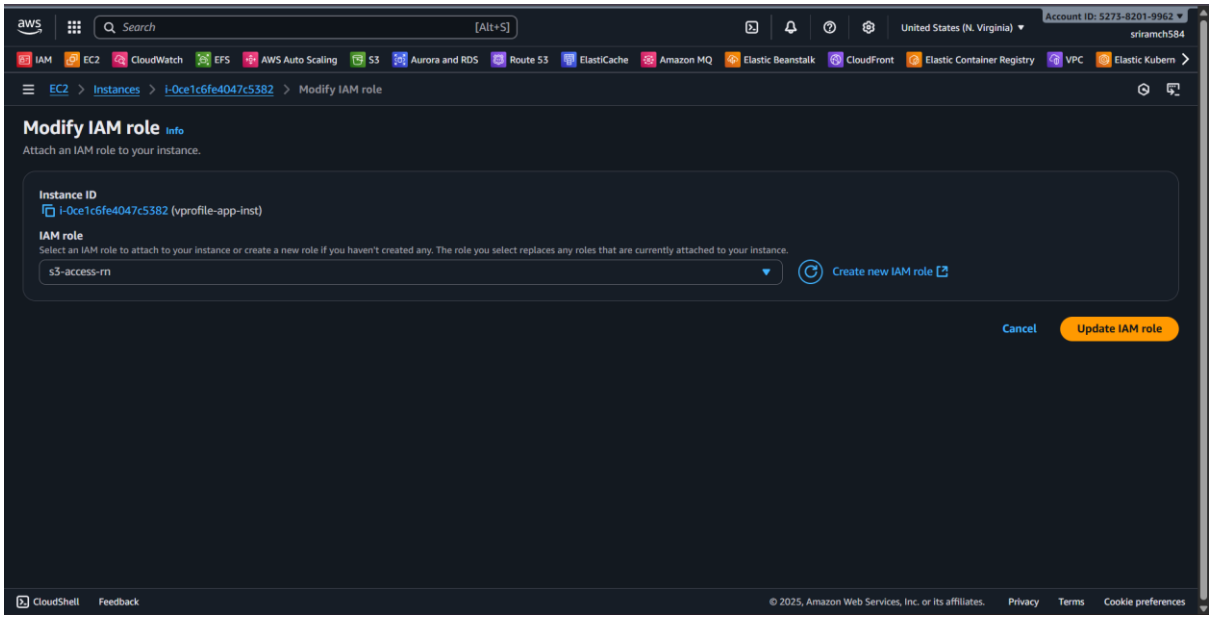
2. **Create IAM user for CLI access**

- Give **S3 Full Access** permission.
- Generate **Access Key & Secret Key** → store safely (CSV download).



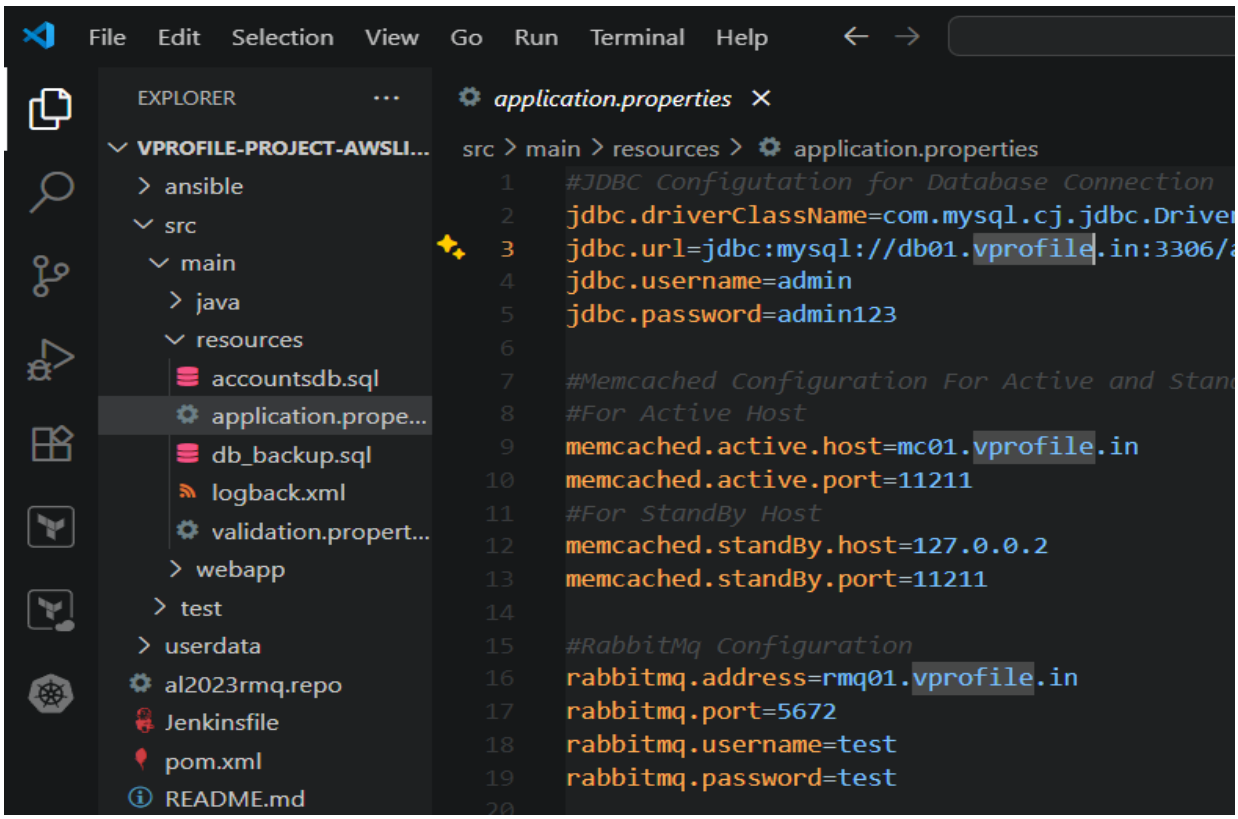
3. **Create IAM role for EC2**

- AWS service: **EC2**
- Attach **S3 Full Access** policy.
- Apply this role to **app01 instance**.



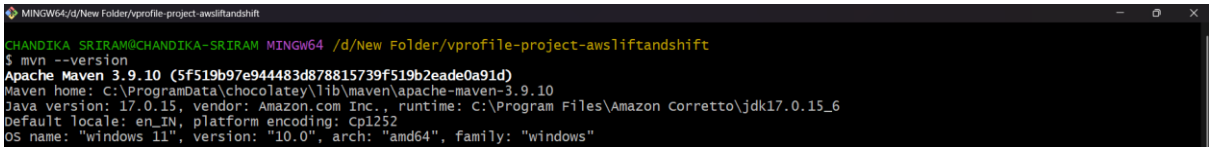
Step 3: Build Artifact

1. Open `application.properties`:
    - Update backend services with **Route 53 DNS names**, not IPs:
        - `db01.vprofile.in`
        - `mc01.vprofile.in`
        - `rmq01.vprofile.in`



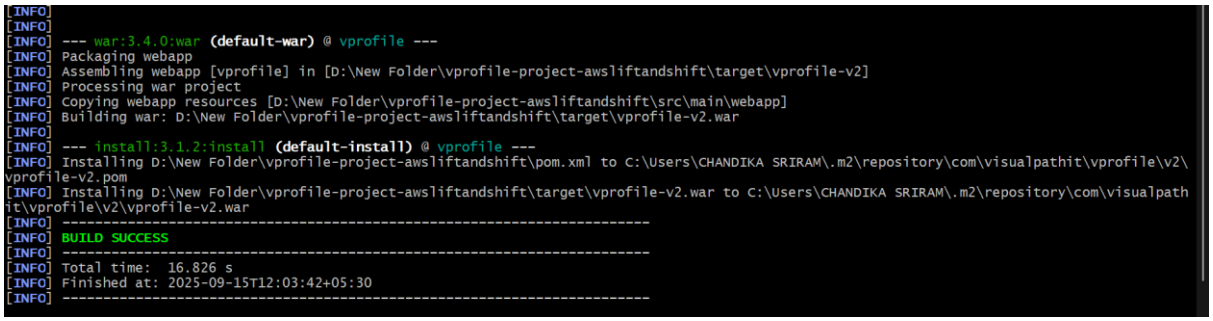
- Validate **Maven, JDK, and AWS CLI** installation:

`mvn –version`

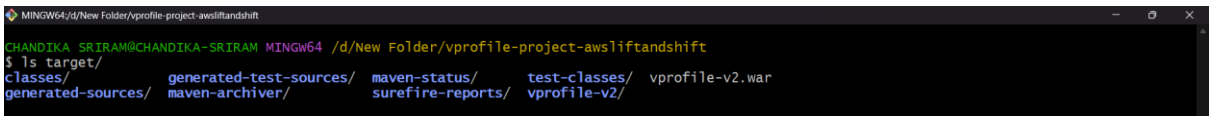


- Build artifact:

`mvn install`



- Artifact generated: `target/vprofile-v`

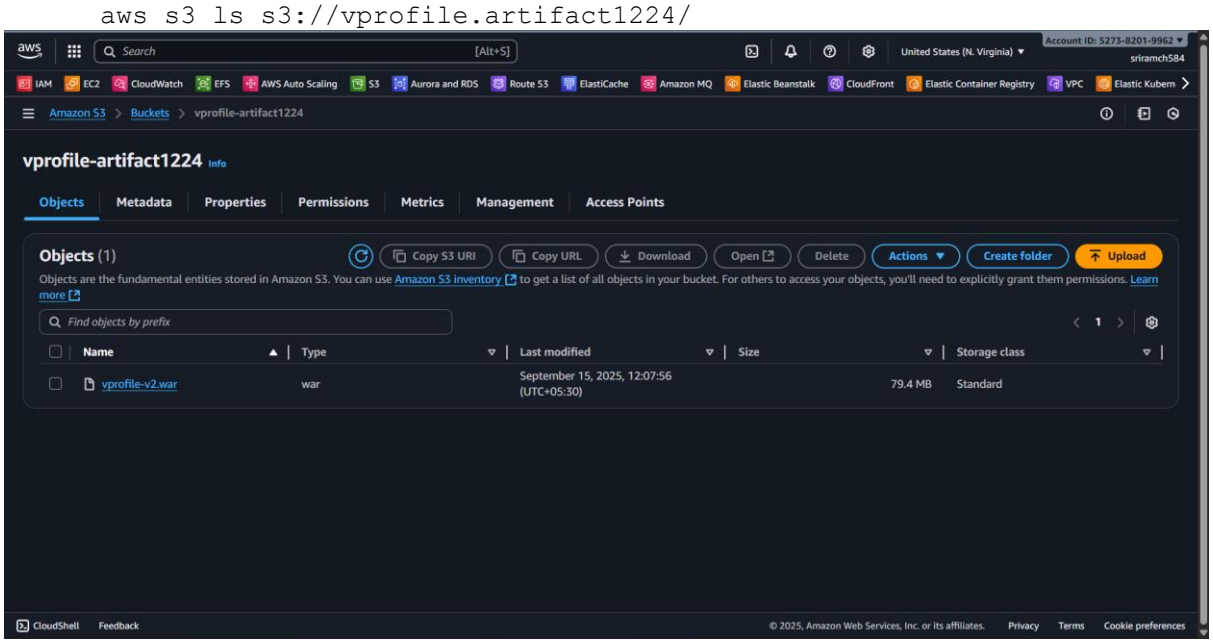


Step 4: Push Artifact to S3

1. Configure AWS CLI with IAM user credentials:
2. `aws configure`
    - Enter Access Key, Secret Key, Default region (e.g., `us-east-1`), Output format (JSON).
3. Push artifact to S3:`aws s3` cp `target/vprofile-v2.war s3://vprofile.artifact1224/`

4. Verify artifact is uploaded:

```
aws s3 ls s3://vprofile.artifact1224/
```



Step 5: Deploy Artifact on Tomcat (app01)

1. SSH into **app01 instance**:

   ```
   ssh -i <key.pem> ubuntu@<public-ip>
   ```

2. Install AWS CLI on instance if needed:

   ```
   snap install aws-cli –classic
   ```

   ```
   root@ip-172-31-22-102:~# snap install aws-cli --classic
   snap "aws-cli" is already installed, see 'snap help refresh'
   root@ip-172-31-22-102:~# aws

   usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
   To see help text, you can run:

     aws help
     aws <command> help
     aws <command> <subcommand> help

   aws: error: the following arguments are required: command
   ```

3. Copy artifact from S3:

   ```
   aws s3 cp s3://vprofile.artifact1224/vprofile-v2.war /tmp/
   ```

   ```
   root@ip-172-31-22-102:~# aws s3 cp s3://vprofile-artifact1224/vprofile-v2.war /tmp/
   download: s3://vprofile-artifact1224/vprofile-v2.war to ../tmp/vprofile-v2.war
   ```

4. Stop Tomcat:

   ```
   systemctl stop tomcat10

   systemctl daemon-reload
   ```

```
root@ip-172-31-22-102:~# systemctl daemon-reload
root@ip-172-31-22-102:~# systemctl stop tomcat10
```

5. Remove default app:

```
rm -rf /var/lib/tomcat10/webapps/ROOT
```

```
root@ip-172-31-22-102:~# systemctl daemon-reload
root@ip-172-31-22-102:~# systemctl stop tomcat10
```

6. Deploy new artifact:

cp `/tmp/vprofile-v2.war /var/lib/tomcat10/webapps/ROOT.war`

```
root@ip-172-31-22-102:~# ls /var/lib/tomcat10/webapps/
ROOT.war
```

7. Start Tomcat:

```
systemctl start tomcat10
```

8. Tomcat extracts .war automatically; app becomes available in a few minutes.

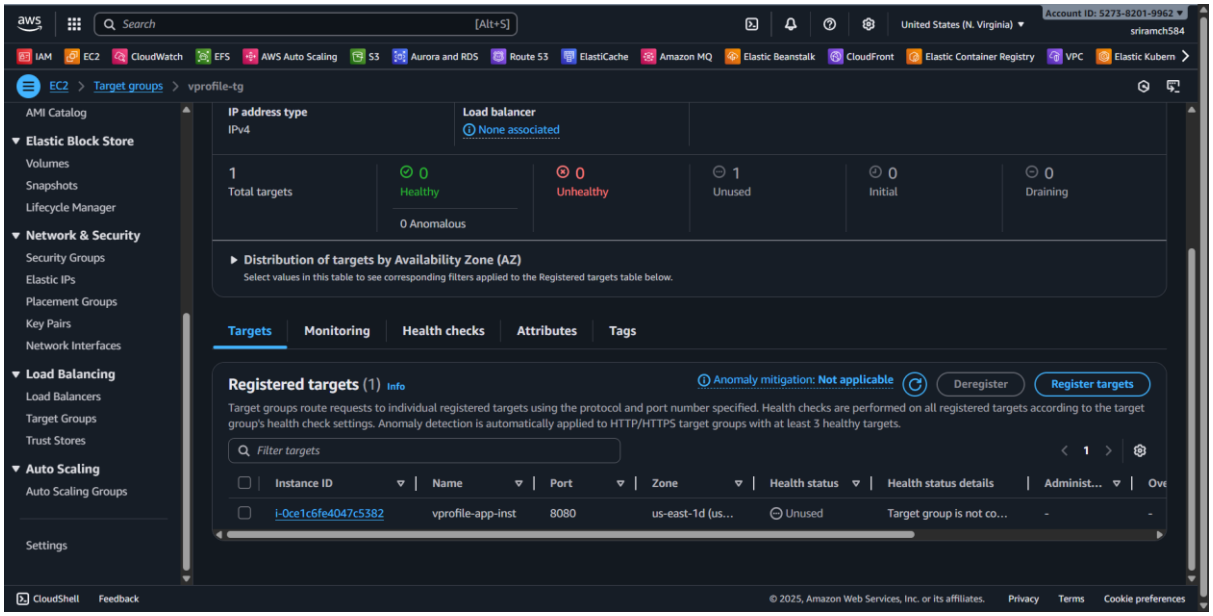
Setup Load Balancer for V-Profile Application

Step 1: Create Target Group

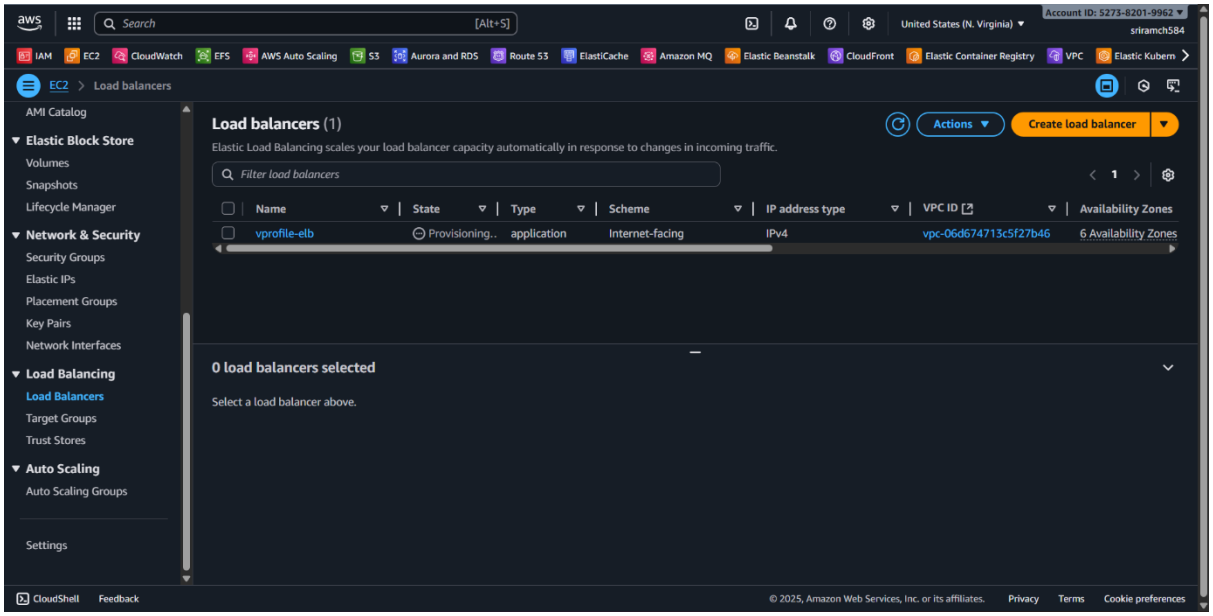
- Go to **Load Balancing → Target Groups → Create Target Group**.
- Type: **Instances**
- Protocol: **HTTP**, Port: **8080** (Tomcat default port)
- Health Check: Override default port 80 → set **8080**
- Add **app01 instance** to the target group.
- ✓ Target group created successfully.

Step 2: Create Load Balancer

- Type: **Application Load Balancer (ALB)** for HTTP and HTTPS traffic.
- Name: `vprofile-elb` (example)
- Network mapping: Select **VPC** and all **Availability Zones**.
- Security group: Select **load balancer security group**.
- Listeners:
    - HTTP → Port 80 → Target group

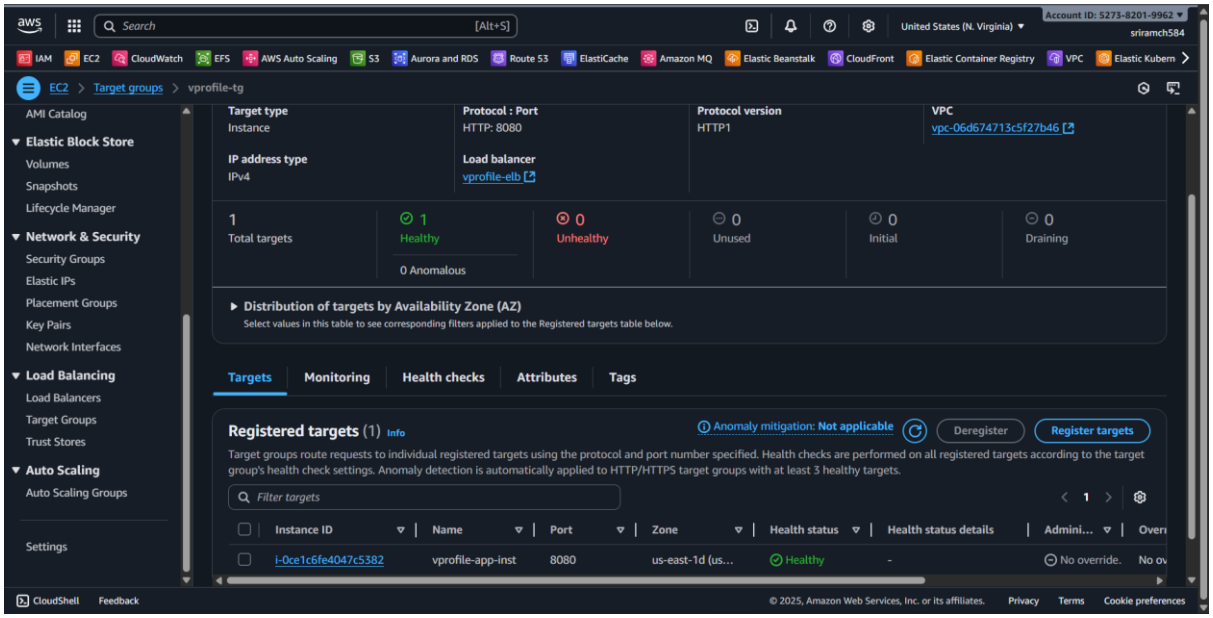


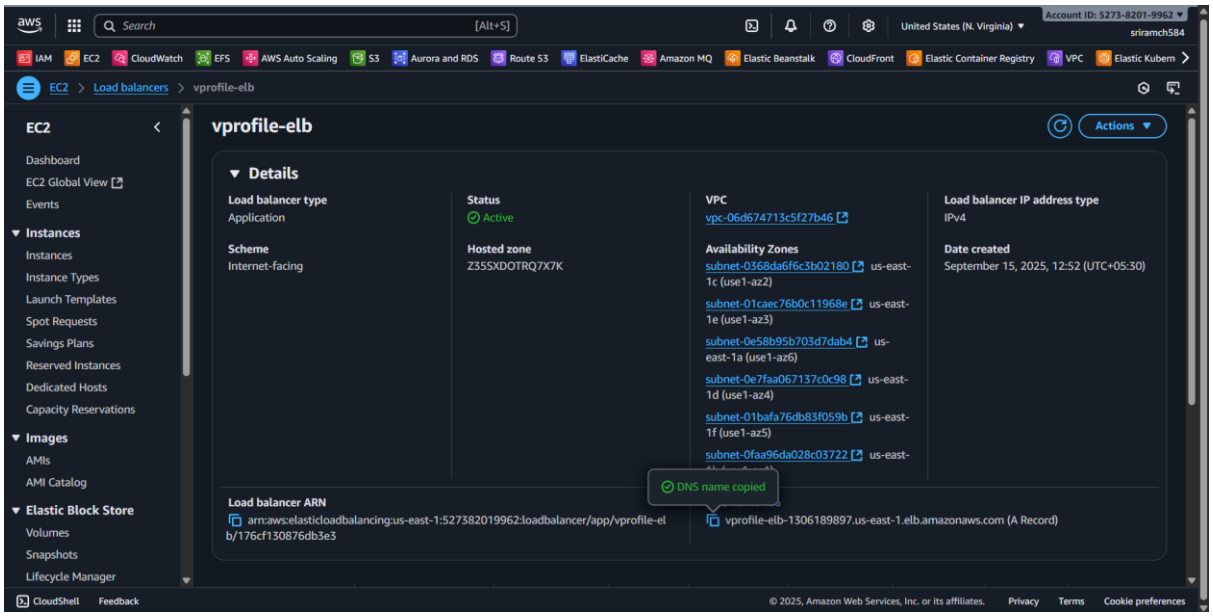
- Click **Create Load Balancer**.



Step 3: Verify Load Balancer & Health

- Check **Target Group → Health Checks**:
  - Instances must be **healthy**.

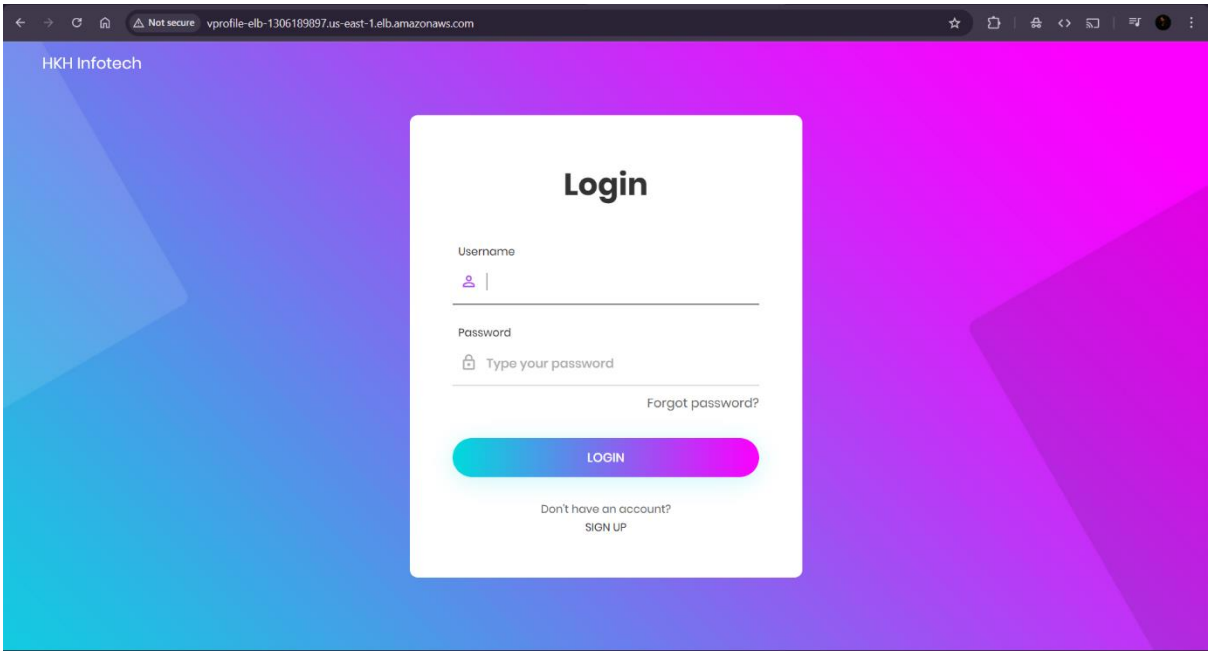




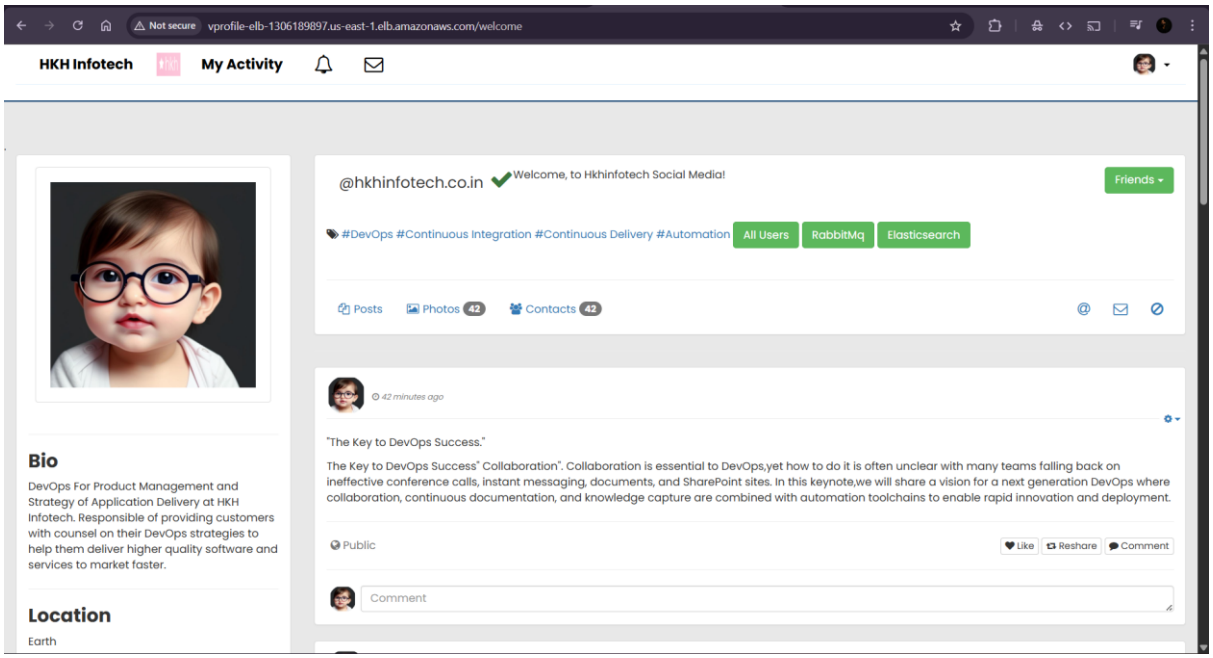
    - Common issues if unhealthy:
        - Tomcat not running on instance
        - Security group does not allow **port 8080** from load balancer
        - Health check port incorrect
- Load Balancer URL: Test **HTTP** (port 80).

Step 4: Verify Application

- Login with **admin credentials**.
- Username: admin_vp
- Password: admin_vp



- Check connections for backend services:
  - **Database**: User info retrieved successfully
  - **RabbitMQ**: Queues generated
  - **Memcache**: Data caching verified



Auto Scaling Group (ASG) for V-Profile Application

Purpose

- Automatically **scale out** instances when load increases.

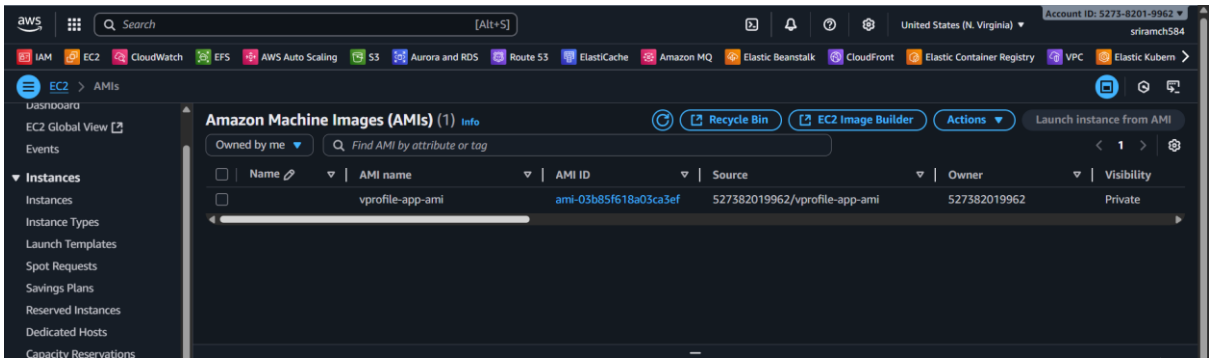
- Automatically **scale in** when load decreases.
- Ensures **high availability** and **performance** of the application.

Three Key Components

1. **AMI (Amazon Machine Image)** – Snapshot of your app instance (`app01`) to launch new instances.
2. **Launch Template** – Configuration for launching instances:
    - AMI to use
    - Instance type (e.g., T2/T3 micro)
    - Security group
    - Key pair
    - IAM role (optional)
    - Tags (e.g., name, project)
3. **Auto Scaling Group** – Defines scaling rules and instance count:
    - Minimum, maximum, and desired number of instances
    - Target group for load balancer
    - Health check settings

Step 1: Create AMI

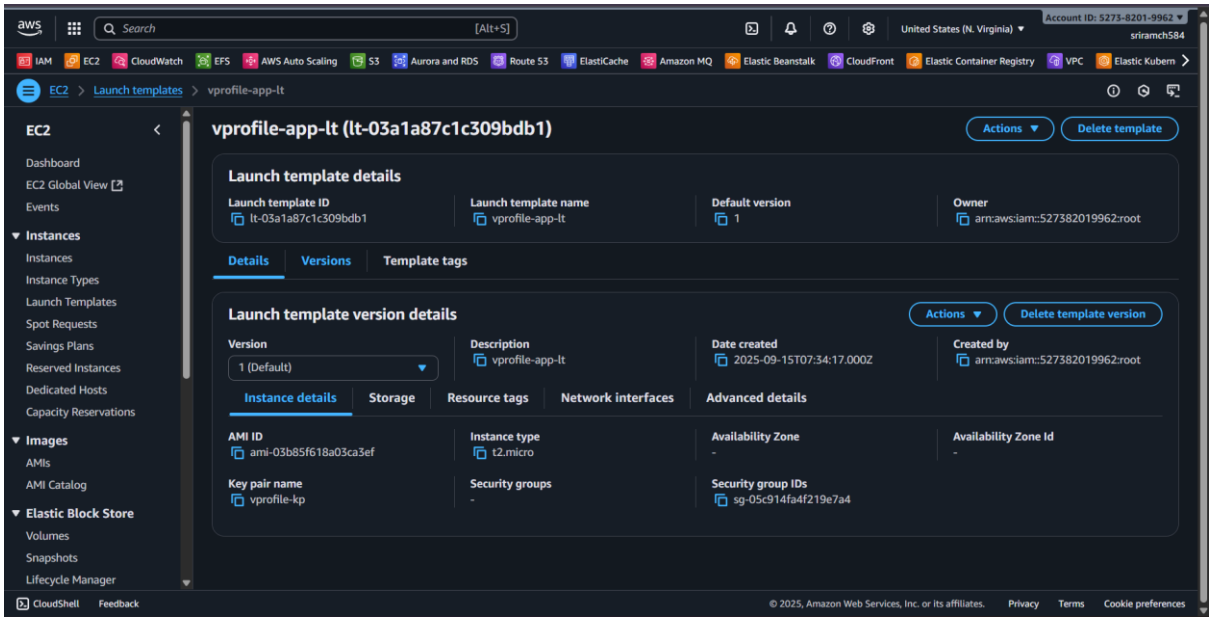
1. Select **app01 instance → Actions → Image & Templates → Create Image**
2. Give a **name** (e.g., `VProfile-App-AMI`) and **description**.
3. Wait for AMI to become **available**.



Step 2: Create Launch Template

1. Go to **Launch Templates → Create Launch Template**
2. Name: `VProfile-LAB-App`
3. Select **AMI** (the one just created)
4. Select **Instance type** (e.g., T2/T3 micro)
5. Select **key pair** for SSH
6. Assign **security group** (App Security Group)

7. Add **tags**:
    - Name: `vprofile-app-01-02`
    - Project: `VProfile`
8. (Optional) Assign **IAM role** for automatic permissions
9. Click **Create Launch Template**

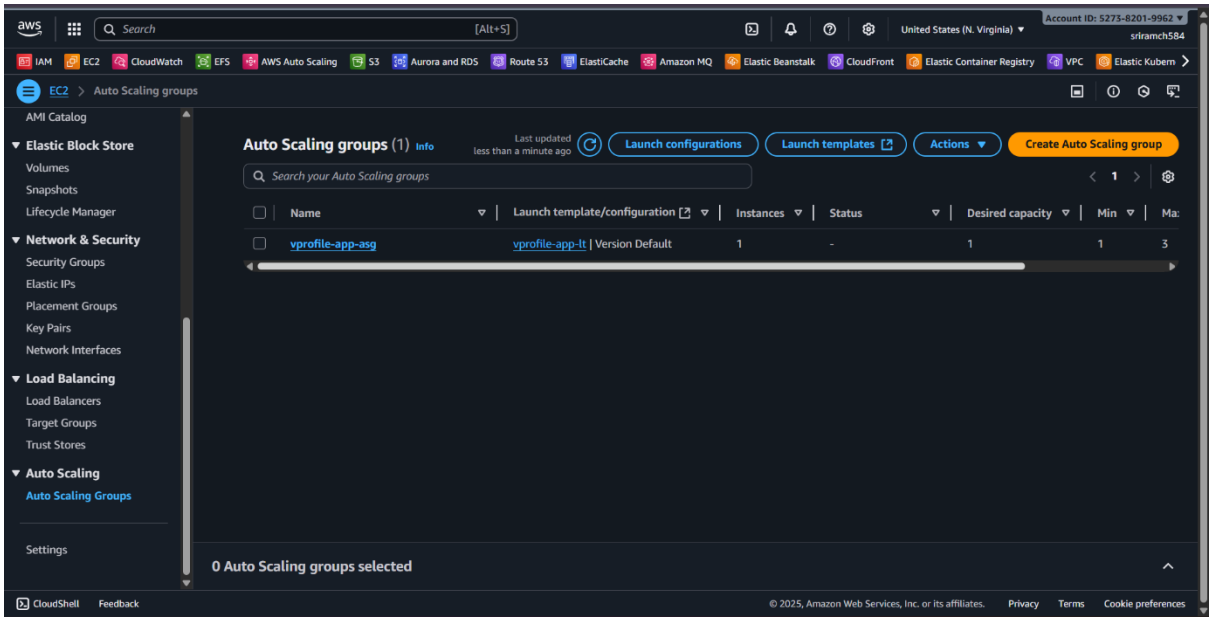


Step 3: Create Auto Scaling Group

1. Go to **Auto Scaling Groups → Create Auto Scaling Group**
2. Name: `VProfile-LAB-ASG`
3. Select **launch template** created above
4. Configure network:
    - Default VPC
    - All Availability Zones (or selected ones)
5. Attach to **existing load balancer → target group**
6. Health check:
    - Enable **Load Balancer health check** → ensures service health is verified
7. Group size:
    - Minimum: 1
    - Desired: 1–4 (can adjust)
    - Maximum: 4
8. Scaling policies:
    - **Target tracking** based on CPU utilization (e.g., scale out if CPU > 50%, scale in if < 50%)
9. Instance scaling protection (optional):
    - Prevents termination of specific instances for troubleshooting
10. Notifications (optional) via **SNS**

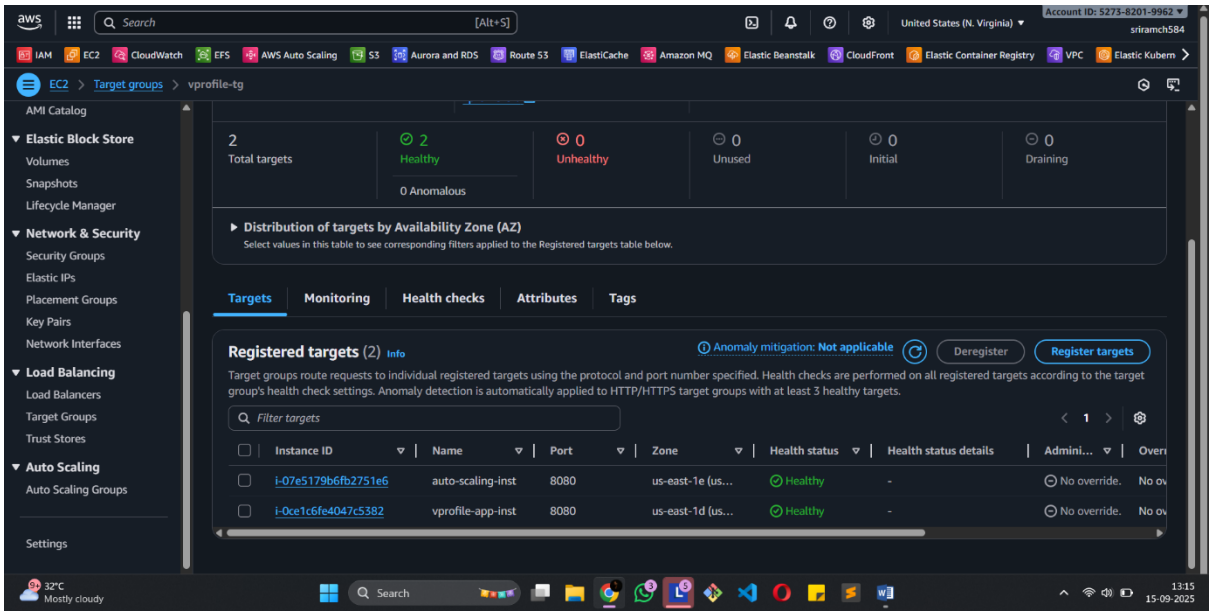
Step 4: Configure Stickiness (Important for V-Profile App)

- Go to **Target Group → Attributes → Edit → Enable stickiness**
- Purpose: Ensure user sessions always connect to the same instance
- Duration: Set as required (e.g., 1 day)
- Why needed: App cannot handle cross-instance authentication without stickiness



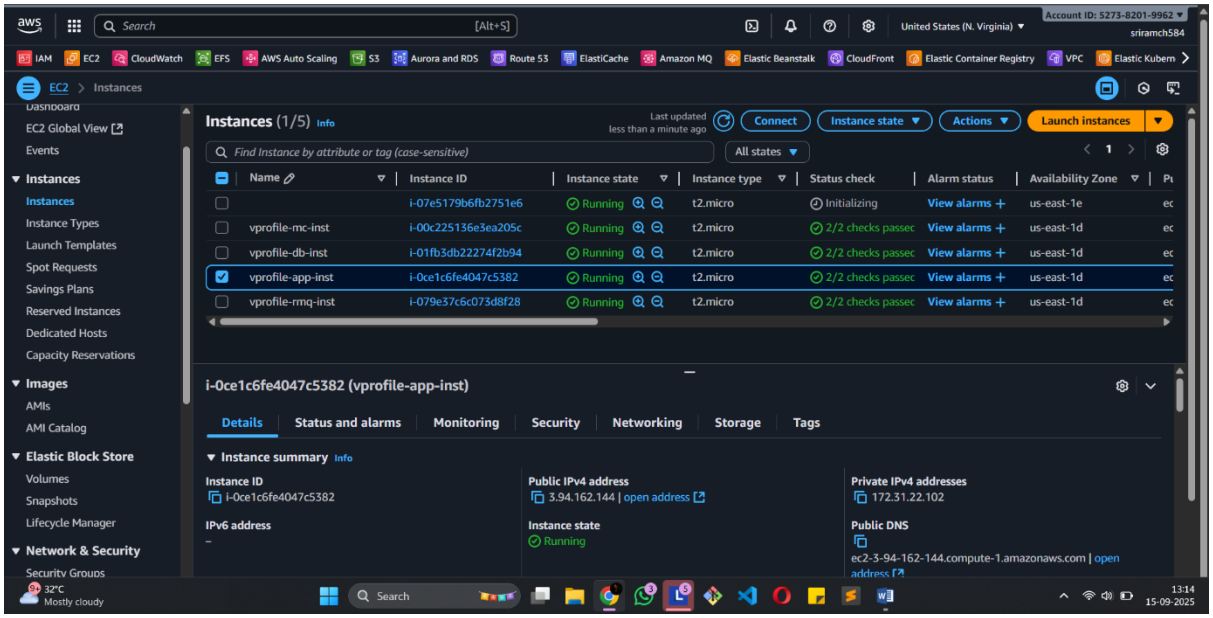
Step 5: Validate ASG

1. ASG launches instance automatically.
2. Check **Target Group → Targets →** instance should be **healthy**.



3. Deregister and terminate any manually added app01 instance:

- ASG will maintain instance count automatically.



4. Test application via **Load Balancer URL** to ensure accessibility.